

Baseline Model Experimentation (Working Notebook)

Purpose of this notebook

This notebook serves as a structured experimentation environment for developing baseline models. It is not intended as a final presentation, but as a working space to explore model behavior, compare approaches, and document intermediate findings.

The focus is on:

- testing multiple model families under consistent conditions,
 - observing how different algorithms respond to the dataset,
 - identifying early signals about model suitability.
-

Why this step matters

Before committing to a final modeling approach, it is necessary to explore a range of models and understand their behavior. This helps avoid premature decisions and provides evidence for selecting a model later in the thesis.

How to use this notebook

This notebook is exploratory but structured. Each step includes short reasoning notes to document:

- why a method is used,
 - what is being observed,
 - and whether it informs a later decision.
-

Expected outcome

The outcome is a set of baseline results and observations that will inform:

- feature engineering,
- model refinement,
- and final model selection.

Imports and Setup

I start by importing the required libraries for data handling, visualization, and modeling.

I grouped the imports based on their purpose to keep the notebook organized and easier to read. This also helps avoid unnecessary imports and makes debugging easier.

At this stage, I only import what is needed for baseline experimentation.

```
In [1]: # Data handling
import pandas as pd
import numpy as np

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Evaluation metrics
from sklearn.metrics import (
    accuracy_score,
    f1_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay
)

# Models
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import LinearSVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.neural_network import MLPClassifier
from xgboost import XGBClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
```

```
In [2]: # Load processed datasets created in Notebook 1
train_df = pd.read_csv("../data/processed/train.csv")
val_df = pd.read_csv("../data/processed/val.csv")
test_df = pd.read_csv("../data/processed/test.csv")

print("Train shape:", train_df.shape)
print("Validation shape:", val_df.shape)
print("Test shape:", test_df.shape)
```

```
Train shape: (69979, 175)
Validation shape: (14995, 175)
Test shape: (14996, 175)
```

```
In [3]: # Define target column
target_column = "disease_encoded"

# Split each dataset into features (X) and target (y)
X_train = train_df.drop(columns=[target_column])
y_train = train_df[target_column]

X_val = val_df.drop(columns=[target_column])
y_val = val_df[target_column]

X_test = test_df.drop(columns=[target_column])
y_test = test_df[target_column]
```

```
print("X_train shape:", X_train.shape)
print("y_train shape:", y_train.shape)
print("X_val shape:", X_val.shape)
print("y_val shape:", y_val.shape)
print("X_test shape:", X_test.shape)
print("y_test shape:", y_test.shape)
```

```
X_train shape: (69979, 174)
y_train shape: (69979,)
X_val shape: (14995, 174)
y_val shape: (14995,)
X_test shape: (14996, 174)
y_test shape: (14996,)
```

In [4]: **def** evaluate_model(model, X_train, y_train, X_val, y_val, model_name):

```
    # Train model
    model.fit(X_train, y_train)

    # Predict on validation set
    y_val_pred = model.predict(X_val)

    # Compute metrics
    accuracy = accuracy_score(y_val, y_val_pred)
    f1_weighted = f1_score(y_val, y_val_pred, average="weighted")
    f1_macro = f1_score(y_val, y_val_pred, average="macro")

    # Store results
    results = {
        "Model": model_name,
        "Accuracy": accuracy,
        "Weighted F1": f1_weighted,
        "Macro F1": f1_macro
    }

    return results, y_val_pred

# Initialize results storage (run once at top of notebook)
results_list = []
```

Dummy Baseline Model

I trained a dummy classifier as a baseline model.

I did this because before evaluating more complex models, it is important to establish a minimum reference point. The dummy model represents a simple strategy (such as predicting the most frequent class) and does not learn meaningful patterns.

From this, I understood that all subsequent models must outperform this baseline to demonstrate that they are capturing useful information from the data.

In [5]: *# Dummy baseline model*
dummy_model = DummyClassifier(strategy="most_frequent", random_state=42)

Train, predict and Evaluate
results, y_pred_dummy = evaluate_model(
 dummy_model,
 X_train, y_train,

```

X_val, y_val,
"Dummy"
)

results_list.append(results)
# Display
results_df = pd.DataFrame(results_list)
display(results_df)

```

	Model	Accuracy	Weighted F1	Macro F1
0	Dummy	0.213004	0.074807	0.009492

Logistic Regression Experiments

I trained two Logistic Regression models:

- one with feature scaling using a pipeline,
- and one without scaling.

I did this because Logistic Regression can be sensitive to feature scale. Scaling may improve optimization and model performance, especially when features are on different ranges.

I also used `class_weight="balanced"` to handle possible class imbalance and ensure the model does not favor majority classes.

From this, I aim to understand whether scaling improves performance or convergence for this dataset.

```

In [6]: # Logistic Regression with scaling
lr_scaled = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", LogisticRegression(
        max_iter=200,
        class_weight="balanced",
        random_state=42
    ))
])

metrics_scaled, y_pred_lr_scaled = evaluate_model(
    lr_scaled,
    X_train, y_train,
    X_val, y_val,
    "Logistic Regression (Scaled)"
)

results_list.append(metrics_scaled)
# Display
results_df = pd.DataFrame(results_list)
display(results_df)

```

	Model	Accuracy	Weighted F1	Macro F1
0	Dummy	0.213004	0.074807	0.009492
1	Logistic Regression (Scaled)	0.333378	0.346475	0.259448

```
In [7]: # Logistic Regression without scaling
lr_unscaled = LogisticRegression(
    max_iter=2600,
    class_weight="balanced",
    random_state=42
)

metrics_unscaled, y_pred_lr_unscaled = evaluate_model(
    lr_unscaled,
    X_train, y_train,
    X_val, y_val,
    "Logistic Regression (Unscaled)"
)

results_list.append(metrics_unscaled)
results_df = pd.DataFrame(results_list)
display(results_df)
```

	Model	Accuracy	Weighted F1	Macro F1
0	Dummy	0.213004	0.074807	0.009492
1	Logistic Regression (Scaled)	0.333378	0.346475	0.259448
2	Logistic Regression (Unscaled)	0.330977	0.344180	0.242567

What I observed

Both scaled and unscaled Logistic Regression models were evaluated using the same metrics.

I also checked the number of iterations required for convergence.

From this, I understood:

- whether scaling improves predictive performance,
- whether scaling helps the model converge faster,
- and whether adding preprocessing is necessary for this dataset.

I compared scaled and unscaled Logistic Regression models.

Both models performed similarly, with the scaled version showing a small improvement across the evaluation metrics, particularly macro F1.

Given that Logistic Regression can be sensitive to feature scale and the scaled version showed slightly more consistent performance, I chose the scaled model as the baseline for further comparison.

Random Forest Experiments

I trained Random Forest models using several parameter settings.

I did this because Random Forest is a strong non-linear model, and its performance can change depending on parameters such as the number of trees, tree depth, and the number of features considered at each split.

I started with a base Random Forest configuration and then changed one parameter group at a time. This helps me understand which settings improve performance and whether additional complexity is useful for this dataset.

From this, I want to identify a Random Forest configuration that provides strong and balanced performance for later comparison with other models.

```
In [8]: # Base RF
rf_model = RandomForestClassifier(
    n_estimators=200,
    random_state=42,
    class_weight="balanced_subsample",
    n_jobs=-1
)

metrics_rf, _ = evaluate_model(
    rf_model,
    X_train, y_train,
    X_val, y_val,
    "RF"
)
results_list.append(metrics_rf)

results_df = pd.DataFrame(results_list)
display(results_df)
```

	Model	Accuracy	Weighted F1	Macro F1
0	Dummy	0.213004	0.074807	0.009492
1	Logistic Regression (Scaled)	0.333378	0.346475	0.259448
2	Logistic Regression (Unscaled)	0.330977	0.344180	0.242567
3	RF	0.396866	0.357490	0.213581

```
In [9]: # RF - n_estimators
for n in [100, 300]:
    model = RandomForestClassifier(
        n_estimators=n,
        random_state=42,
        class_weight="balanced_subsample",
        n_jobs=-1
    )

    metrics, _ = evaluate_model(
        model,
        X_train, y_train,
        X_val, y_val,
        f"RF_n{n}"
    )
    results_list.append(metrics)
```

```
results_df = pd.DataFrame(results_list)
display(results_df)
```

	Model	Accuracy	Weighted F1	Macro F1
0	Dummy	0.213004	0.074807	0.009492
1	Logistic Regression (Scaled)	0.333378	0.346475	0.259448
2	Logistic Regression (Unscaled)	0.330977	0.344180	0.242567
3	RF	0.396866	0.357490	0.213581
4	RF_n100	0.390130	0.351630	0.215175
5	RF_n300	0.404335	0.366329	0.225271

```
In [10]: # RF - max_depth
for depth in [None, 10, 20]:
    model = RandomForestClassifier(
        n_estimators=200,
        max_depth=depth,
        random_state=42,
        class_weight="balanced_subsample",
        n_jobs=-1
    )

    metrics, _ = evaluate_model(
        model,
        X_train, y_train,
        X_val, y_val,
        f"RF_depth_{depth}"
    )
    results_list.append(metrics)

results_df = pd.DataFrame(results_list)
display(results_df)
```

	Model	Accuracy	Weighted F1	Macro F1
0	Dummy	0.213004	0.074807	0.009492
1	Logistic Regression (Scaled)	0.333378	0.346475	0.259448
2	Logistic Regression (Unscaled)	0.330977	0.344180	0.242567
3	RF	0.396866	0.357490	0.213581
4	RF_n100	0.390130	0.351630	0.215175
5	RF_n300	0.404335	0.366329	0.225271
6	RF_depth_None	0.396866	0.357490	0.213581
7	RF_depth_10	0.161521	0.162333	0.156807
8	RF_depth_20	0.334712	0.339322	0.231148

```
In [11]: # RF - max_features
for mf in ["sqrt", "log2"]:
    model = RandomForestClassifier(
        n_estimators=200,
```

```

        max_features=mf,
        random_state=42,
        class_weight="balanced_subsample",
        n_jobs=-1
    )

    metrics, _ = evaluate_model(
        model,
        X_train, y_train,
        X_val, y_val,
        f"RF_maxfeat_{mf}"
    )
    results_list.append(metrics)

results_df = pd.DataFrame(results_list)
display(results_df)

```

	Model	Accuracy	Weighted F1	Macro F1
0	Dummy	0.213004	0.074807	0.009492
1	Logistic Regression (Scaled)	0.333378	0.346475	0.259448
2	Logistic Regression (Unscaled)	0.330977	0.344180	0.242567
3	RF	0.396866	0.357490	0.213581
4	RF_n100	0.390130	0.351630	0.215175
5	RF_n300	0.404335	0.366329	0.225271
6	RF_depth_None	0.396866	0.357490	0.213581
7	RF_depth_10	0.161521	0.162333	0.156807
8	RF_depth_20	0.334712	0.339322	0.231148
9	RF_maxfeat_sqrt	0.396866	0.357490	0.213581
10	RF_maxfeat_log2	0.382994	0.336301	0.167604

What I observed

I evaluated multiple Random Forest configurations by changing the number of trees, maximum depth, and feature selection.

Increasing the number of trees improved performance slightly, with RF_n300 performing better than RF_n100.

Limiting tree depth too aggressively (depth=10) significantly reduced performance, indicating underfitting. A moderate depth (depth=20) provided better results.

Changing max_features did not improve performance, with "log2" performing worse than the default setting.

Overall, the best Random Forest configuration was RF_depth_20.

What I understood

From this, I understood that:

- Random Forest performance is sensitive to hyperparameters.
- Increasing complexity helps up to a point, but too much restriction leads to underfitting.
- The model benefits from sufficient depth and a reasonable number of trees.

However, even the best Random Forest model did not outperform Logistic Regression.

This suggests that the dataset may be better suited to simpler models, and that complex non-linear models do not necessarily provide an advantage in this case.

K-Nearest Neighbors (KNN) Experiments

I trained multiple KNN models with different values of k and weighting strategies.

I did this because KNN is a distance-based model, and its performance depends on how many neighbors are considered and how their distances are used.

I used feature scaling because KNN is sensitive to feature magnitude, and unscaled features can distort distance calculations.

From this, I aim to understand how the choice of k and weighting affects model performance and whether KNN is suitable for this dataset.

```
In [12]: # KNN with k=5 (baseline KNN)
knn_model = Pipeline([
    ("scaler", StandardScaler()), # scaling required for distance-based models
    ("clf", KNeighborsClassifier(n_neighbors=5, n_jobs=-1))
])

metrics_knn, _ = evaluate_model(
    knn_model,
    X_train, y_train,
    X_val, y_val,
    "KNN_k5"
)

results_list.append(metrics_knn)

# display updated results sorted by Macro F1
results_df = pd.DataFrame(results_list).sort_values(by="Macro F1", ascending=False)
display(results_df)
```

	Model	Accuracy	Weighted F1	Macro F1
1	Logistic Regression (Scaled)	0.333378	0.346475	0.259448
2	Logistic Regression (Unscaled)	0.330977	0.344180	0.242567
8	RF_depth_20	0.334712	0.339322	0.231148
5	RF_n300	0.404335	0.366329	0.225271
4	RF_n100	0.390130	0.351630	0.215175
3	RF	0.396866	0.357490	0.213581
6	RF_depth_None	0.396866	0.357490	0.213581
9	RF_maxfeat_sqrt	0.396866	0.357490	0.213581
11	KNN_k5	0.292831	0.277303	0.182631
10	RF_maxfeat_log2	0.382994	0.336301	0.167604
7	RF_depth_10	0.161521	0.162333	0.156807
0	Dummy	0.213004	0.074807	0.009492

```
In [13]: # smaller k → more local decision boundary (can increase variance)
knn_model_1 = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", KNeighborsClassifier(n_neighbors=3, n_jobs=-1))
])

metrics_knn_1, _ = evaluate_model(
    knn_model_1,
    X_train, y_train,
    X_val, y_val,
    "KNN_k3"
)

results_list.append(metrics_knn_1)

results_df = pd.DataFrame(results_list).sort_values(by="Macro F1", ascending=True)
display(results_df)
```

	Model	Accuracy	Weighted F1	Macro F1
1	Logistic Regression (Scaled)	0.333378	0.346475	0.259448
2	Logistic Regression (Unscaled)	0.330977	0.344180	0.242567
8	RF_depth_20	0.334712	0.339322	0.231148
5	RF_n300	0.404335	0.366329	0.225271
4	RF_n100	0.390130	0.351630	0.215175
3	RF	0.396866	0.357490	0.213581
6	RF_depth_None	0.396866	0.357490	0.213581
9	RF_maxfeat_sqrt	0.396866	0.357490	0.213581
11	KNN_k5	0.292831	0.277303	0.182631
10	RF_maxfeat_log2	0.382994	0.336301	0.167604
12	KNN_k3	0.270023	0.252810	0.167214
7	RF_depth_10	0.161521	0.162333	0.156807
0	Dummy	0.213004	0.074807	0.009492

```
In [14]: # larger k → smoother decision boundary (can reduce variance)
knn_model_2 = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", KNeighborsClassifier(n_neighbors=7, n_jobs=-1))
])

metrics_knn_2, _ = evaluate_model(
    knn_model_2,
    X_train, y_train,
    X_val, y_val,
    "KNN_k7"
)

results_list.append(metrics_knn_2)

results_df = pd.DataFrame(results_list).sort_values(by="Macro F1", ascending=False)
display(results_df)
```

	Model	Accuracy	Weighted F1	Macro F1
1	Logistic Regression (Scaled)	0.333378	0.346475	0.259448
2	Logistic Regression (Unscaled)	0.330977	0.344180	0.242567
8	RF_depth_20	0.334712	0.339322	0.231148
5	RF_n300	0.404335	0.366329	0.225271
4	RF_n100	0.390130	0.351630	0.215175
3	RF	0.396866	0.357490	0.213581
6	RF_depth_None	0.396866	0.357490	0.213581
9	RF_maxfeat_sqrt	0.396866	0.357490	0.213581
13	KNN_k7	0.299100	0.281679	0.184078
11	KNN_k5	0.292831	0.277303	0.182631
10	RF_maxfeat_log2	0.382994	0.336301	0.167604
12	KNN_k3	0.270023	0.252810	0.167214
7	RF_depth_10	0.161521	0.162333	0.156807
0	Dummy	0.213004	0.074807	0.009492

```
In [15]: # distance weighting → closer neighbors have more influence
knn_model_3 = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", KNeighborsClassifier(
        n_neighbors=5,
        weights="distance",
        n_jobs=-1
    ))
])

metrics_knn_3, _ = evaluate_model(
    knn_model_3,
    X_train, y_train,
    X_val, y_val,
    "KNN_k5_distance"
)

results_list.append(metrics_knn_3)

results_df = pd.DataFrame(results_list).sort_values(by="Macro F1", ascending=True)
display(results_df)
```

	Model	Accuracy	Weighted F1	Macro F1
1	Logistic Regression (Scaled)	0.333378	0.346475	0.259448
2	Logistic Regression (Unscaled)	0.330977	0.344180	0.242567
8	RF_depth_20	0.334712	0.339322	0.231148
5	RF_n300	0.404335	0.366329	0.225271
4	RF_n100	0.390130	0.351630	0.215175
3	RF	0.396866	0.357490	0.213581
6	RF_depth_None	0.396866	0.357490	0.213581
9	RF_maxfeat_sqrt	0.396866	0.357490	0.213581
14	KNN_k5_distance	0.289897	0.283893	0.184506
13	KNN_k7	0.299100	0.281679	0.184078
11	KNN_k5	0.292831	0.277303	0.182631
10	RF_maxfeat_log2	0.382994	0.336301	0.167604
12	KNN_k3	0.270023	0.252810	0.167214
7	RF_depth_10	0.161521	0.162333	0.156807
0	Dummy	0.213004	0.074807	0.009492

What I observed

I evaluated multiple KNN models with different values of k and weighting strategies.

Among the KNN models:

- KNN_k5_distance achieved the best performance (Macro F1 $\approx$ 0.185)
- KNN_k7 and KNN_k5 performed similarly but slightly lower
- KNN_k3 performed the worst among KNN models

Overall, increasing k slightly improved stability, while distance weighting provided a small improvement.

However, all KNN models performed worse than both Logistic Regression and Random Forest models.

What I understood

From this, I understood that:

- KNN is sensitive to the choice of k and benefits slightly from distance weighting.
- Smaller values of k (e.g., k=3) lead to worse performance, likely due to higher variance.
- Increasing k improves stability but does not significantly improve overall performance.

Most importantly, KNN did not perform as well as Logistic Regression or Random Forest.

This suggests that the dataset may not be well-suited for distance-based methods, and that the feature space may not support strong nearest-neighbor relationships.

Therefore, KNN is not selected as a primary model for this problem.

Support Vector Machine (SVM) Experiments

I trained multiple linear SVM models with different regularization strengths (C values).

I did this because SVM performance depends on the balance between model complexity and regularization.

I used feature scaling because SVM is sensitive to feature magnitude.

From this, I want to understand how changing the regularization parameter affects model performance and whether SVM is suitable for this dataset.

```
In [16]: # SVM (linear, scaled)
svm_model = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", LinearSVC(
        class_weight="balanced",
        max_iter=5000,
        random_state=42
    ))
])

metrics_svm, _ = evaluate_model(
    svm_model,
    X_train, y_train,
    X_val, y_val,
    "SVM_linear"
)

results_list.append(metrics_svm)

results_df = pd.DataFrame(results_list).sort_values(by="Macro F1", ascending=True)
display(results_df)
```

	Model	Accuracy	Weighted F1	Macro F1
1	Logistic Regression (Scaled)	0.333378	0.346475	0.259448
2	Logistic Regression (Unscaled)	0.330977	0.344180	0.242567
15	SVM_linear	0.323041	0.337076	0.235941
8	RF_depth_20	0.334712	0.339322	0.231148
5	RF_n300	0.404335	0.366329	0.225271
4	RF_n100	0.390130	0.351630	0.215175
3	RF	0.396866	0.357490	0.213581
6	RF_depth_None	0.396866	0.357490	0.213581
9	RF_maxfeat_sqrt	0.396866	0.357490	0.213581
14	KNN_k5_distance	0.289897	0.283893	0.184506
13	KNN_k7	0.299100	0.281679	0.184078
11	KNN_k5	0.292831	0.277303	0.182631
10	RF_maxfeat_log2	0.382994	0.336301	0.167604
12	KNN_k3	0.270023	0.252810	0.167214
7	RF_depth_10	0.161521	0.162333	0.156807
0	Dummy	0.213004	0.074807	0.009492

```
In [17]: # smaller C → stronger regularization
svm_model_1 = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", LinearSVC(
        C=0.1,
        class_weight="balanced",
        max_iter=3000,
        random_state=42
    ))
])

metrics_svm_1, _ = evaluate_model(
    svm_model_1,
    X_train, y_train,
    X_val, y_val,
    "SVM_C0.1"
)

results_list.append(metrics_svm_1)

results_df = pd.DataFrame(results_list).sort_values(by="Macro F1", ascending=True)
display(results_df)
```

	Model	Accuracy	Weighted F1	Macro F1
1	Logistic Regression (Scaled)	0.333378	0.346475	0.259448
2	Logistic Regression (Unscaled)	0.330977	0.344180	0.242567
15	SVM_linear	0.323041	0.337076	0.235941
8	RF_depth_20	0.334712	0.339322	0.231148
16	SVM_C0.1	0.320240	0.335222	0.228331
5	RF_n300	0.404335	0.366329	0.225271
4	RF_n100	0.390130	0.351630	0.215175
6	RF_depth_None	0.396866	0.357490	0.213581
9	RF_maxfeat_sqrt	0.396866	0.357490	0.213581
3	RF	0.396866	0.357490	0.213581
14	KNN_k5_distance	0.289897	0.283893	0.184506
13	KNN_k7	0.299100	0.281679	0.184078
11	KNN_k5	0.292831	0.277303	0.182631
10	RF_maxfeat_log2	0.382994	0.336301	0.167604
12	KNN_k3	0.270023	0.252810	0.167214
7	RF_depth_10	0.161521	0.162333	0.156807
0	Dummy	0.213004	0.074807	0.009492

```
In [18]: # default regularization level
svm_model_2 = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", LinearSVC(
        C=1,
        class_weight="balanced",
        max_iter=3000,
        random_state=42
    ))
])

metrics_svm_2, _ = evaluate_model(
    svm_model_2,
    X_train, y_train,
    X_val, y_val,
    "SVM_C1"
)

results_list.append(metrics_svm_2)

results_df = pd.DataFrame(results_list).sort_values(by="Macro F1", ascending=True)
display(results_df)
```


	Model	Accuracy	Weighted F1	Macro F1
1	Logistic Regression (Scaled)	0.333378	0.346475	0.259448
2	Logistic Regression (Unscaled)	0.330977	0.344180	0.242567
17	SVM_C1	0.323041	0.337076	0.235941
15	SVM_linear	0.323041	0.337076	0.235941
8	RF_depth_20	0.334712	0.339322	0.231148
16	SVM_C0.1	0.320240	0.335222	0.228331
5	RF_n300	0.404335	0.366329	0.225271
4	RF_n100	0.390130	0.351630	0.215175
9	RF_maxfeat_sqrt	0.396866	0.357490	0.213581
6	RF_depth_None	0.396866	0.357490	0.213581
3	RF	0.396866	0.357490	0.213581
14	KNN_k5_distance	0.289897	0.283893	0.184506
13	KNN_k7	0.299100	0.281679	0.184078
11	KNN_k5	0.292831	0.277303	0.182631
10	RF_maxfeat_log2	0.382994	0.336301	0.167604
12	KNN_k3	0.270023	0.252810	0.167214
7	RF_depth_10	0.161521	0.162333	0.156807
0	Dummy	0.213004	0.074807	0.009492

```
In [19]: # weaker regularization → more flexible model
svm_model_3 = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", LinearSVC(
        C=10,
        class_weight="balanced",
        max_iter=5000,
        random_state=42
    ))
])

metrics_svm_3, _ = evaluate_model(
    svm_model_3,
    X_train, y_train,
    X_val, y_val,
    "SVM_C10"
)

results_list.append(metrics_svm_3)

results_df = pd.DataFrame(results_list).sort_values(by="Macro F1", ascending=True)
display(results_df)
```

```
/opt/anaconda3/lib/python3.12/site-packages/sklearn/svm/_base.py:1249: ConvergenceWarning: Liblinear failed to converge, increase the number of iterations.
  warnings.warn(
```

	Model	Accuracy	Weighted F1	Macro F1
1	Logistic Regression (Scaled)	0.333378	0.346475	0.259448
2	Logistic Regression (Unscaled)	0.330977	0.344180	0.242567
17	SVM_C1	0.323041	0.337076	0.235941
15	SVM_linear	0.323041	0.337076	0.235941
18	SVM_C10	0.323508	0.337453	0.232713
8	RF_depth_20	0.334712	0.339322	0.231148
16	SVM_C0.1	0.320240	0.335222	0.228331
5	RF_n300	0.404335	0.366329	0.225271
4	RF_n100	0.390130	0.351630	0.215175
9	RF_maxfeat_sqrt	0.396866	0.357490	0.213581
6	RF_depth_None	0.396866	0.357490	0.213581
3	RF	0.396866	0.357490	0.213581
14	KNN_k5_distance	0.289897	0.283893	0.184506
13	KNN_k7	0.299100	0.281679	0.184078
11	KNN_k5	0.292831	0.277303	0.182631
10	RF_maxfeat_log2	0.382994	0.336301	0.167604
12	KNN_k3	0.270023	0.252810	0.167214
7	RF_depth_10	0.161521	0.162333	0.156807
0	Dummy	0.213004	0.074807	0.009492

What I observed

I evaluated multiple SVM models with different regularization strengths (C values).

The performance across SVM models was relatively consistent:

- SVM_C1 and the scaled linear SVM achieved similar performance (Macro F1 $\approx$ 0.236)
- Increasing C to 10 did not improve performance and slightly reduced Macro F1
- Reducing C to 0.1 also led to lower performance

Overall, changing the regularization parameter did not significantly change the results.

What I understood

From this, I understood that:

- SVM performance is relatively stable across different regularization settings for this dataset.
- Increasing model flexibility (higher C) does not improve performance, suggesting that the model is not limited by underfitting.

- Stronger regularization (lower C) slightly reduces performance, indicating that the model still requires some flexibility.

When compared to other models, SVM performs similarly to Random Forest but does not outperform Logistic Regression.

This suggests that a linear decision boundary is sufficient for this dataset, but Logistic Regression provides a slightly better fit.

XGBoost Experiments

I trained multiple XGBoost models with different configurations.

I did this because XGBoost performance depends on parameters such as tree depth, learning rate, and number of trees.

I started with a baseline configuration and then tested variations by:

- reducing tree depth to create a simpler model,
- decreasing the learning rate and increasing the number of trees for more gradual learning,
- increasing tree depth to allow a more flexible model.

Unlike KNN and SVM, I did not apply feature scaling because XGBoost is a tree-based model and does not depend on feature magnitude.

From this, I aim to understand whether boosting-based models improve performance compared to Logistic Regression and Random Forest.

```
In [20]: # Base XGBoost configuration
xgb_model = XGBClassifier(
    n_estimators=200,
    max_depth=6,
    learning_rate=0.1,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42,
    eval_metric="mlogloss"
)

metrics_xgb, _ = evaluate_model(
    xgb_model,
    X_train, y_train,
    X_val, y_val,
    "XGBoost"
)

results_list.append(metrics_xgb)

results_df = pd.DataFrame(results_list).sort_values(by="Macro F1", ascending=True)
display(results_df)
```

	Model	Accuracy	Weighted F1	Macro F1
19	XGBoost	0.647082	0.637245	0.392429
1	Logistic Regression (Scaled)	0.333378	0.346475	0.259448
2	Logistic Regression (Unscaled)	0.330977	0.344180	0.242567
17	SVM_C1	0.323041	0.337076	0.235941
15	SVM_linear	0.323041	0.337076	0.235941
18	SVM_C10	0.323508	0.337453	0.232713
8	RF_depth_20	0.334712	0.339322	0.231148
16	SVM_C0.1	0.320240	0.335222	0.228331
5	RF_n300	0.404335	0.366329	0.225271
4	RF_n100	0.390130	0.351630	0.215175
3	RF	0.396866	0.357490	0.213581
6	RF_depth_None	0.396866	0.357490	0.213581
9	RF_maxfeat_sqrt	0.396866	0.357490	0.213581
14	KNN_k5_distance	0.289897	0.283893	0.184506
13	KNN_k7	0.299100	0.281679	0.184078
11	KNN_k5	0.292831	0.277303	0.182631
10	RF_maxfeat_log2	0.382994	0.336301	0.167604
12	KNN_k3	0.270023	0.252810	0.167214
7	RF_depth_10	0.161521	0.162333	0.156807
0	Dummy	0.213004	0.074807	0.009492

```
In [21]: # Simpler XGBoost with shallower trees
xgb_model_1 = XGBClassifier(
    n_estimators=200,
    max_depth=4,
    learning_rate=0.1,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42,
    eval_metric="mlogloss"
)

metrics_xgb_1, _ = evaluate_model(
    xgb_model_1,
    X_train, y_train,
    X_val, y_val,
    "XGBoost_depth4"
)

results_list.append(metrics_xgb_1)

results_df = pd.DataFrame(results_list).sort_values(by="Macro F1", ascending=True)
display(results_df)
```

	Model	Accuracy	Weighted F1	Macro F1
19	XGBoost	0.647082	0.637245	0.392429
20	XGBoost_depth4	0.603668	0.592488	0.362304
1	Logistic Regression (Scaled)	0.333378	0.346475	0.259448
2	Logistic Regression (Unscaled)	0.330977	0.344180	0.242567
15	SVM_linear	0.323041	0.337076	0.235941
17	SVM_C1	0.323041	0.337076	0.235941
18	SVM_C10	0.323508	0.337453	0.232713
8	RF_depth_20	0.334712	0.339322	0.231148
16	SVM_C0.1	0.320240	0.335222	0.228331
5	RF_n300	0.404335	0.366329	0.225271
4	RF_n100	0.390130	0.351630	0.215175
6	RF_depth_None	0.396866	0.357490	0.213581
9	RF_maxfeat_sqrt	0.396866	0.357490	0.213581
3	RF	0.396866	0.357490	0.213581
14	KNN_k5_distance	0.289897	0.283893	0.184506
13	KNN_k7	0.299100	0.281679	0.184078
11	KNN_k5	0.292831	0.277303	0.182631
10	RF_maxfeat_log2	0.382994	0.336301	0.167604
12	KNN_k3	0.270023	0.252810	0.167214
7	RF_depth_10	0.161521	0.162333	0.156807
0	Dummy	0.213004	0.074807	0.009492

```
In [22]: # Smaller learning rate with more trees
xgb_model_2 = XGBClassifier(
    n_estimators=300,
    max_depth=6,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42,
    eval_metric="mlogloss"
)

metrics_xgb_2, _ = evaluate_model(
    xgb_model_2,
    X_train, y_train,
    X_val, y_val,
    "XGBoost_lr0.05_n300"
)

results_list.append(metrics_xgb_2)

results_df = pd.DataFrame(results_list).sort_values(by="Macro F1", ascending=False)
display(results_df)
```

	Model	Accuracy	Weighted F1	Macro F1
19	XGBoost	0.647082	0.637245	0.392429
21	XGBoost_lr0.05_n300	0.627276	0.615745	0.369793
20	XGBoost_depth4	0.603668	0.592488	0.362304
1	Logistic Regression (Scaled)	0.333378	0.346475	0.259448
2	Logistic Regression (Unscaled)	0.330977	0.344180	0.242567
17	SVM_C1	0.323041	0.337076	0.235941
15	SVM_linear	0.323041	0.337076	0.235941
18	SVM_C10	0.323508	0.337453	0.232713
8	RF_depth_20	0.334712	0.339322	0.231148
16	SVM_C0.1	0.320240	0.335222	0.228331
5	RF_n300	0.404335	0.366329	0.225271
4	RF_n100	0.390130	0.351630	0.215175
9	RF_maxfeat_sqrt	0.396866	0.357490	0.213581
6	RF_depth_None	0.396866	0.357490	0.213581
3	RF	0.396866	0.357490	0.213581
14	KNN_k5_distance	0.289897	0.283893	0.184506
13	KNN_k7	0.299100	0.281679	0.184078
11	KNN_k5	0.292831	0.277303	0.182631
10	RF_maxfeat_log2	0.382994	0.336301	0.167604
12	KNN_k3	0.270023	0.252810	0.167214
7	RF_depth_10	0.161521	0.162333	0.156807
0	Dummy	0.213004	0.074807	0.009492

```
In [23]: # Slightly more flexible XGBoost
xgb_model_3 = XGBClassifier(
    n_estimators=300,
    max_depth=8,
    learning_rate=0.1,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42,
    eval_metric="mlogloss"
)

metrics_xgb_3, _ = evaluate_model(
    xgb_model_3,
    X_train, y_train,
    X_val, y_val,
    "XGBoost_depth8_n300"
)

results_list.append(metrics_xgb_3)
```

```
results_df = pd.DataFrame(results_list).sort_values(by="Macro F1", ascending=False)
display(results_df)
```

	Model	Accuracy	Weighted F1	Macro F1
22	XGBoost_depth8_n300	0.661621	0.651510	0.418387
19	XGBoost	0.647082	0.637245	0.392429
21	XGBoost_lr0.05_n300	0.627276	0.615745	0.369793
20	XGBoost_depth4	0.603668	0.592488	0.362304
1	Logistic Regression (Scaled)	0.333378	0.346475	0.259448
2	Logistic Regression (Unscaled)	0.330977	0.344180	0.242567
15	SVM_linear	0.323041	0.337076	0.235941
17	SVM_C1	0.323041	0.337076	0.235941
18	SVM_C10	0.323508	0.337453	0.232713
8	RF_depth_20	0.334712	0.339322	0.231148
16	SVM_C0.1	0.320240	0.335222	0.228331
5	RF_n300	0.404335	0.366329	0.225271
4	RF_n100	0.390130	0.351630	0.215175
6	RF_depth_None	0.396866	0.357490	0.213581
3	RF	0.396866	0.357490	0.213581
9	RF_maxfeat_sqrt	0.396866	0.357490	0.213581
14	KNN_k5_distance	0.289897	0.283893	0.184506
13	KNN_k7	0.299100	0.281679	0.184078
11	KNN_k5	0.292831	0.277303	0.182631
10	RF_maxfeat_log2	0.382994	0.336301	0.167604
12	KNN_k3	0.270023	0.252810	0.167214
7	RF_depth_10	0.161521	0.162333	0.156807
0	Dummy	0.213004	0.074807	0.009492

What I observed

I evaluated multiple XGBoost models with different configurations, including variations in tree depth, learning rate, and number of trees.

Among all models tested, XGBoost clearly achieved the best performance.

- The best model was XGBoost_depth8_n300 with Macro F1 ≈ 0.418
- The baseline XGBoost model also performed strongly (Macro F1 ≈ 0.392)
- Reducing the learning rate and increasing the number of trees slightly reduced performance
- Using a smaller tree depth (depth=4) resulted in lower performance compared to deeper trees

Overall, increasing model complexity through deeper trees and more estimators improved performance.

What I understood

From this, I understood that:

- XGBoost is highly effective for this dataset and significantly outperforms all other models tested.
- Increasing model capacity (deeper trees and more estimators) allows the model to better capture complex patterns in the data.
- Simpler configurations (shallower trees or lower learning rate) do not perform as well, indicating that the dataset benefits from higher model complexity.

This suggests that the dataset contains non-linear relationships that are better captured by boosting-based models.

Therefore, XGBoost is the most suitable model for this problem.

Neural Network Experiments

I trained multiple neural network models with different hidden layer sizes and one regularized configuration.

I did this because neural network performance depends strongly on architecture and regularization. Changing the hidden layer sizes allows me to test whether a smaller or larger network is more suitable for this dataset.

I also tested a regularized version by increasing the alpha value to examine whether stronger regularization improves generalization.

I used feature scaling because neural networks are sensitive to feature magnitude, and I enabled early stopping to reduce unnecessary training and limit overfitting.

From this, I want to understand whether neural networks provide an advantage over the other models tested so far.

```
In [24]: # baseline neural network with two hidden layers
nn_model_128_64 = Pipeline([
    ("scaler", StandardScaler()), # scaling is important for neural network
    ("clf", MLPClassifier(
        hidden_layer_sizes=(128, 64),
        max_iter=200,
        random_state=42,
        early_stopping=True, # stop training when validation performance
        validation_fraction=0.1,
        n_iter_no_change=10
    ))
])
```



```

metrics_nn_128_64, _ = evaluate_model(
    nn_model_128_64,
    X_train, y_train,
    X_val, y_val,
    "NN_(128,64)"
)

results_list.append(metrics_nn_128_64)

# display updated ranking after adding this model
results_df = pd.DataFrame(results_list).sort_values(by="Macro F1", ascending=False)
display(results_df)

```

	Model	Accuracy	Weighted F1	Macro F1
22	XGBoost_depth8_n300	0.661621	0.651510	0.418387
19	XGBoost	0.647082	0.637245	0.392429
21	XGBoost_lr0.05_n300	0.627276	0.615745	0.369793
20	XGBoost_depth4	0.603668	0.592488	0.362304
23	NN_(128,64)	0.531911	0.521264	0.291543
1	Logistic Regression (Scaled)	0.333378	0.346475	0.259448
2	Logistic Regression (Unscaled)	0.330977	0.344180	0.242567
15	SVM_linear	0.323041	0.337076	0.235941
17	SVM_C1	0.323041	0.337076	0.235941
18	SVM_C10	0.323508	0.337453	0.232713
8	RF_depth_20	0.334712	0.339322	0.231148
16	SVM_C0.1	0.320240	0.335222	0.228331
5	RF_n300	0.404335	0.366329	0.225271
4	RF_n100	0.390130	0.351630	0.215175
6	RF_depth_None	0.396866	0.357490	0.213581
9	RF_maxfeat_sqrt	0.396866	0.357490	0.213581
3	RF	0.396866	0.357490	0.213581
14	KNN_k5_distance	0.289897	0.283893	0.184506
13	KNN_k7	0.299100	0.281679	0.184078
11	KNN_k5	0.292831	0.277303	0.182631
10	RF_maxfeat_log2	0.382994	0.336301	0.167604
12	KNN_k3	0.270023	0.252810	0.167214
7	RF_depth_10	0.161521	0.162333	0.156807
0	Dummy	0.213004	0.074807	0.009492

```

In [25]: # smaller network to test whether a simpler architecture is sufficient
nn_model_64_32 = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", MLPClassifier(

```

```

        hidden_layer_sizes=(64, 32),
        max_iter=200,
        random_state=42,
        early_stopping=True,
        validation_fraction=0.1,
        n_iter_no_change=10
    ))
])

metrics_nn_64_32, _ = evaluate_model(
    nn_model_64_32,
    X_train, y_train,
    X_val, y_val,
    "NN_(64,32)"
)

results_list.append(metrics_nn_64_32)

results_df = pd.DataFrame(results_list).sort_values(by="Macro F1", ascending=False)
display(results_df)

```

	Model	Accuracy	Weighted F1	Macro F1
22	XGBoost_depth8_n300	0.661621	0.651510	0.418387
19	XGBoost	0.647082	0.637245	0.392429
21	XGBoost_lr0.05_n300	0.627276	0.615745	0.369793
20	XGBoost_depth4	0.603668	0.592488	0.362304
24	NN_(64,32)	0.502968	0.495420	0.293907
23	NN_(128,64)	0.531911	0.521264	0.291543
1	Logistic Regression (Scaled)	0.333378	0.346475	0.259448
2	Logistic Regression (Unscaled)	0.330977	0.344180	0.242567
15	SVM_linear	0.323041	0.337076	0.235941
17	SVM_C1	0.323041	0.337076	0.235941
18	SVM_C10	0.323508	0.337453	0.232713
8	RF_depth_20	0.334712	0.339322	0.231148
16	SVM_C0.1	0.320240	0.335222	0.228331
5	RF_n300	0.404335	0.366329	0.225271
4	RF_n100	0.390130	0.351630	0.215175
3	RF	0.396866	0.357490	0.213581
9	RF_maxfeat_sqrt	0.396866	0.357490	0.213581
6	RF_depth_None	0.396866	0.357490	0.213581
14	KNN_k5_distance	0.289897	0.283893	0.184506
13	KNN_k7	0.299100	0.281679	0.184078
11	KNN_k5	0.292831	0.277303	0.182631
10	RF_maxfeat_log2	0.382994	0.336301	0.167604
12	KNN_k3	0.270023	0.252810	0.167214
7	RF_depth_10	0.161521	0.162333	0.156807
0	Dummy	0.213004	0.074807	0.009492

```
In [26]: # larger network to test whether additional capacity improves performance
nn_model_256_128 = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", MLPClassifier(
        hidden_layer_sizes=(256, 128),
        max_iter=200,
        random_state=42,
        early_stopping=True,
        validation_fraction=0.1,
        n_iter_no_change=10
    ))
])

metrics_nn_256_128, _ = evaluate_model(
    nn_model_256_128,
```

```

X_train, y_train,
X_val, y_val,
"NN_(256,128)"
)

results_list.append(metrics_nn_256_128)

results_df = pd.DataFrame(results_list).sort_values(by="Macro F1", ascending
display(results_df)

```

	Model	Accuracy	Weighted F1	Macro F1
22	XGBoost_depth8_n300	0.661621	0.651510	0.418387
19	XGBoost	0.647082	0.637245	0.392429
21	XGBoost_lr0.05_n300	0.627276	0.615745	0.369793
20	XGBoost_depth4	0.603668	0.592488	0.362304
25	NN_(256,128)	0.551517	0.545278	0.307126
24	NN_(64,32)	0.502968	0.495420	0.293907
23	NN_(128,64)	0.531911	0.521264	0.291543
1	Logistic Regression (Scaled)	0.333378	0.346475	0.259448
2	Logistic Regression (Unscaled)	0.330977	0.344180	0.242567
15	SVM_linear	0.323041	0.337076	0.235941
17	SVM_C1	0.323041	0.337076	0.235941
18	SVM_C10	0.323508	0.337453	0.232713
8	RF_depth_20	0.334712	0.339322	0.231148
16	SVM_C0.1	0.320240	0.335222	0.228331
5	RF_n300	0.404335	0.366329	0.225271
4	RF_n100	0.390130	0.351630	0.215175
6	RF_depth_None	0.396866	0.357490	0.213581
9	RF_maxfeat_sqrt	0.396866	0.357490	0.213581
3	RF	0.396866	0.357490	0.213581
14	KNN_k5_distance	0.289897	0.283893	0.184506
13	KNN_k7	0.299100	0.281679	0.184078
11	KNN_k5	0.292831	0.277303	0.182631
10	RF_maxfeat_log2	0.382994	0.336301	0.167604
12	KNN_k3	0.270023	0.252810	0.167214
7	RF_depth_10	0.161521	0.162333	0.156807
0	Dummy	0.213004	0.074807	0.009492

```

In [27]: # stronger regularization to test whether it improves generalization
nn_model_128_64_alpha = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", MLPClassifier(

```

```

        hidden_layer_sizes=(128, 64),
        alpha=0.01,
        max_iter=200,
        random_state=42,
        early_stopping=True,
        validation_fraction=0.1,
        n_iter_no_change=10
    ))
])

metrics_nn_128_64_alpha, _ = evaluate_model(
    nn_model_128_64_alpha,
    X_train, y_train,
    X_val, y_val,
    "NN_(128,64)_alpha0.01"
)

results_list.append(metrics_nn_128_64_alpha)

results_df = pd.DataFrame(results_list).sort_values(by="Macro F1", ascending=False)
display(results_df)

```

	Model	Accuracy	Weighted F1	Macro F1
22	XGBoost_depth8_n300	0.661621	0.651510	0.418387
19	XGBoost	0.647082	0.637245	0.392429
21	XGBoost_lr0.05_n300	0.627276	0.615745	0.369793
20	XGBoost_depth4	0.603668	0.592488	0.362304
25	NN_(256,128)	0.551517	0.545278	0.307126
26	NN_(128,64)_alpha0.01	0.530110	0.519439	0.296772
24	NN_(64,32)	0.502968	0.495420	0.293907
23	NN_(128,64)	0.531911	0.521264	0.291543
1	Logistic Regression (Scaled)	0.333378	0.346475	0.259448
2	Logistic Regression (Unscaled)	0.330977	0.344180	0.242567
15	SVM_linear	0.323041	0.337076	0.235941
17	SVM_C1	0.323041	0.337076	0.235941
18	SVM_C10	0.323508	0.337453	0.232713
8	RF_depth_20	0.334712	0.339322	0.231148
16	SVM_C0.1	0.320240	0.335222	0.228331
5	RF_n300	0.404335	0.366329	0.225271
4	RF_n100	0.390130	0.351630	0.215175
6	RF_depth_None	0.396866	0.357490	0.213581
9	RF_maxfeat_sqrt	0.396866	0.357490	0.213581
3	RF	0.396866	0.357490	0.213581
14	KNN_k5_distance	0.289897	0.283893	0.184506
13	KNN_k7	0.299100	0.281679	0.184078
11	KNN_k5	0.292831	0.277303	0.182631
10	RF_maxfeat_log2	0.382994	0.336301	0.167604
12	KNN_k3	0.270023	0.252810	0.167214
7	RF_depth_10	0.161521	0.162333	0.156807
0	Dummy	0.213004	0.074807	0.009492

What I observed

I evaluated multiple neural network models with different hidden layer sizes and one regularized configuration.

Among the neural network models:

- NN_(256,128) achieved the best performance (Macro F1 $\approx$ 0.307)
- NN_(128,64) and its regularized version (alpha=0.01) performed similarly but slightly lower

- NN_(64,32) showed the lowest performance among the neural network models

Overall, increasing the network size improved performance, while adding regularization ($\alpha=0.01$) did not lead to improvement.

What I understood

From this, I understood that:

- Neural network performance improves with increased model capacity, as larger architectures are able to capture more complex patterns.
- Smaller networks are likely underfitting the data, leading to lower performance.
- Additional regularization did not improve results, suggesting that overfitting was not a major issue for this setup.

When compared to other models, neural networks performed better than Logistic Regression, SVM, and Random Forest, but did not outperform XGBoost.

This suggests that while the dataset contains complex patterns that benefit from non-linear models, boosting-based methods (XGBoost) are more effective than neural networks for this problem.

```
In [28]: # save final model comparison results for reproducibility and reporting
results_df.to_csv("../results/model_experimentation_comparison_results.csv",
print("Model experimentation comparison results saved successfully.")
```

Model experimentation comparison results saved successfully.

Conclusion

This notebook established a structured baseline for evaluating multiple machine learning algorithms on the given multiclass classification problem. The objective was not only to compare performance, but to understand how different model families respond to the dataset under consistent experimental conditions.

The results demonstrate a clear performance hierarchy across model types. The dummy classifier confirmed a low baseline (Macro F1 ≈ 0.01), validating that meaningful learning is required for this task. Linear models such as Logistic Regression and SVM provided moderate performance (Macro F1 ≈ 0.24 – 0.26), indicating that linear decision boundaries capture some signal but are insufficient to fully model the underlying data complexity.

Tree-based models, particularly Random Forest, showed improved performance compared to linear models but remained limited (Macro F1 ≈ 0.23), suggesting that while non-linearity is important, bagging alone is not enough to capture the full structure of the dataset. Distance-based methods (KNN) consistently underperformed, indicating that the feature space does not support strong local similarity patterns.

Neural networks demonstrated a noticeable improvement (Macro F1 ≈ 0.29 – 0.31), confirming the presence of complex, non-linear relationships. However, their

performance remained below that of boosting-based methods, despite increased architectural capacity.

XGBoost significantly outperformed all other models, achieving the highest performance (Macro F1 ≈ 0.418 with the best configuration). This indicates that the dataset benefits from sequential learning, feature interactions, and higher model capacity. The improvement from shallow to deeper trees further reinforces that capturing complex patterns is critical for this task.

From a modeling perspective, these findings suggest that:

- The dataset contains substantial non-linear structure
- Linear models are insufficient as final solutions but useful as baselines
- Boosting-based methods provide the best balance of bias and variance
- Increasing model complexity is beneficial up to a point, particularly for XGBoost

Based on these results, XGBoost is selected as the primary candidate for further development. Future work should focus on:

- systematic hyperparameter optimization
- feature engineering and feature selection
- handling class imbalance more explicitly
- evaluating performance on the test set and ensuring generalization

Overall, this experimentation phase successfully established a strong empirical foundation for model selection and provides clear direction for subsequent model refinement and optimization.